
When Does Recurrent Memory Help? Six Failure Modes and a Methodology for Honest Evaluation of Augmented LLMs

Anonymous Author(s)

Affiliation

Address

email

Abstract

We began with a simple question: why did a recurrent memory module sometimes look useful, sometimes invisible, and sometimes actively harmful on the same long-conversation callback task? In a 16-version campaign on Memory Residuals — a slot-attention memory matrix attached to Qwen3-0.6B and Qwen3-1.7B — the answer was not a single bug. It was an evaluation problem. Token-averaged chain evaluation diluted a localized callback gain by roughly two orders of magnitude; frozen and joint training measured different causal pathways; the synthetic D4v2 callback template leaked a 32-way category prior that joint-trained models matched directly; more than half of the D4v2 callback windows exposed evidence to the LM path; evidence redaction had to be audited before a zero lift could be trusted; and a random-ID corpus removed the prior but exposed a writer objective that still did not force chain-specific bindings into M_c . We turn these failures into a six-item methodology: callback-aware evaluation, evidence-redacted memory floors, empirical and base-model prior audits, window-leakage counts, a test-time readout audit that asks whether the writer encoded the answer at all, and a held-out-corpus pivot that breaks the templated-data confound. Applying the methodology end-to-end yields a single positive recipe — a frozen pretrained backbone with a fixed-size jointly-trained M_c , supervised by LM-NLL alone — that delivers $+1.32 \pm 0.53$ nats of chain-specific callback CE improvement on LongMemEval-S validation across four seeds at Qwen3-0.6B, with a chain-shuffle confound statistically zero. The methodology rejects all of our earlier published-looking results.

1 Introduction

A practical conversational agent should remember a user across sessions. Three architectural routes for that capability dominate the contemporary literature: *joint training*, in which a memory module and the language-model backbone are trained together end-to-end; *frozen backbone*, in which a small recurrent state is added to a pretrained model whose parameters are then held fixed; and *retrieval*, in which past sessions are indexed and re-fed through the attention window at query time. Papers in each of the three programs report success. Papers from the three programs rarely fail in the same way. And papers from the three programs rarely report the same evaluation controls.

This paper contains a finding that does not look like a finding when described in any one of the three vocabularies. In a 16-version post-mortem of *Memory Residuals* — a slot-attention recurrent memory module attached to Qwen3-0.6B and Qwen3-1.7B that we are able to train under all three regimes on the same architecture — we encountered six independent failure modes. Each is sufficient by itself to make a published-style result look positive when the result is not actually carrying chain-specific information. Five of the six have been observed individually, in one or another of the three research

36 programs, but never together; the sixth (writer-objective failure) is, to our knowledge, novel as a
 37 documented memory-side pathology. The failure modes are independent: fixing one does not fix the
 38 others.

39 The diagnostic state of the literature is that each program checks roughly the failure modes the others
 40 do not. A joint-training paper typically reports task accuracy and a no-memory baseline, which
 41 dodges token-averaging dilution but leaves the joint-training-invisibility mode and the template-
 42 prior leak unmeasured. A frozen-backbone paper typically reports a chain-CE delta, which exposes
 43 the joint-invisibility mode by construction but masks token-averaging dilution and rarely reports
 44 an evidence-redacted floor. A retrieval paper typically reports a recall@ k and a downstream-QA
 45 accuracy, which catches some surface-form leakage but does not address the chain-shuffle confound.
 46 Cross-program, no single paper to our knowledge reports all six controls; most report two or fewer
 47 (Table 1).

48 **Contributions.** This paper makes three contributions and one empirical surprise.

- 49 1. **A field-level diagnosis.** Three research programs of recurrent memory (joint training, frozen
 50 backbone, retrieval) cluster into three evaluation conventions whose reporting matrices do
 51 not overlap. We describe and tabulate the gap (§2).
- 52 2. **Six failure modes (§5).** We catalogue six independent ways a recurrent-memory result
 53 can look positive without carrying chain-specific information; each is load-bearing on at
 54 least one ideology, each is detected by a single cheap diagnostic, and we exhibit a worked
 55 numerical example from the campaign for each.
- 56 3. **Six-item audit methodology (§7).** Callback-aware evaluation, evidence-redacted memory
 57 floors, empirical and base-model prior audits, window-leakage counts, a test-time readout
 58 audit, and a held-out-corpus pivot. Total compute cost < 1 H100-hour per checkpoint,
 59 applies to all three ideologies.
- 60 4. **An empirical surprise (§6).** The auxiliary readout-probe loss that the joint-training literature
 61 canonically prescribes for the writer-objective failure mode is, in our setting, *harmful*:
 62 removing it multiplies Δ_{nm-m} by $\sim 8\times$ at both 0.6B and 1.7B, with a chain-level paired
 63 permutation test giving $p < 10^{-4}$. The recipe that survives all six audits is the simpler,
 64 probe-free one (§8).

65 §9 lays out which of the six failure modes generalize by construction (those that live in the metric or
 66 the corpus, not the architecture) and which we observe only in our specific slot-attention recipe at
 67 $\leq 1.7B$.

68 2 Related Work and Three Ideologies of Recurrent Memory

69 We use the word *ideology* in the modest sense of a research program with its own characteristic
 70 evaluation conventions; nothing in this paper rests on a stronger philosophical reading. The three
 71 programs are not interchangeable engineering choices. Each makes a different bet about where the
 72 chain-specific binding lives, each is exposed to a different subset of the failure modes catalogued in
 73 §5, and each tends to report a different subset of controls.

74 **Joint training.** The bet is that gradient descent over the entire model will allocate credit to
 75 whichever path carries the chain-specific binding. Examples: AutoCompressors [11], Memorising
 76 Transformers [8], TRIME [12], RIMs [18], and the differentiable-memory family (NTM/DNC [15]).
 77 This program is exposed by construction to *joint-training invisibility* (FM2, §5.2) and to *template-*
 78 *prior leak* (FM3, §5.3): when there is more than one path through which the LM head can solve the
 79 callback, gradient descent allocates credit to the easier one, and the memory channel can collapse to
 80 noise.

81 **Frozen backbone.** The bet is that freezing the LM head removes the non-memory paths by
 82 construction; if memory helps, the help has to come through M_c . Examples: the Recurrent Memory
 83 Transformer in its frozen-LM ablation [9, 10], the Block-Recurrent Transformer [13], and the
 84 Compressive Transformer [14]. The Memory Residuals substrate we use falls in this program. This
 85 program escapes FM2/FM3 but is heavily exposed to *token-averaged dilution* (FM1, §5.1) — a frozen

Table 1: Reporting matrix. Of the six diagnostics catalogued in §5, which are reported in representative papers from each ideology and from the long-context-evaluation cluster. ✓ = reported with quantitative evidence; ~ = partially addressed (mentioned, implicitly controlled, or addressed for a related concern); – = not reported. Columns: “CB-CE” callback-aware CE (vs aggregate); “Floor” evidence-redacted memory floor; “Prior” empirical or base-model prior baseline; “WLk” window-leakage count; “Red.” redaction validity audit; “TTT” test-time readout audit. Cell judgments follow the per-paper notes in our literature audit (Appendix E). No paper in the audit reports more than two of the six diagnostics with quantitative evidence.

Paper	Ideology	CB-CE	Floor	Prior	WLk	Red.	TTT
Memorising Transformers [8]	joint	–	–	–	–	–	–
AutoCompressors [11]	joint	–	~	–	–	–	–
TRIME [12]	joint	–	–	~	–	–	–
RIMs [18]	joint	–	–	–	–	–	–
NTM/DNC [15]	joint	–	–	–	–	–	–
RMT-frozen [9]	frozen	–	–	–	–	–	–
ARMT [10]	frozen	–	–	–	–	–	–
Block-Recurrent [13]	frozen	–	–	–	–	–	–
Compressive Tr. [14]	frozen	–	–	–	–	–	–
RAG [16]	retrieval	–	–	–	~	–	–
DOS-RAG [17]	retrieval	–	–	~	~	–	–
NoLiMa [1]	long-ctx	–	–	✓	✓	–	–
RULER [2]	long-ctx	–	–	✓	✓	–	–
Lost-in-the-Middle [3]	long-ctx	–	–	~	–	–	–
LongBench [4]	long-ctx	~	–	✓	–	–	–
BABILong [5]	long-ctx	–	–	~	~	–	–
ZeroSCROLLS [19]	long-ctx	–	–	~	–	–	–
LongMemEval [6]	long-ctx	~	–	~	~	–	~
This paper	all three	✓	✓	✓	✓	✓	✓

86 backbone’s CE delta is small relative to the score window — and to *redaction uncertainty* (FM5,
87 §5.5): a zero evidence-lift could mean either a content-blind writer or a leaky redaction definition.
88 Without an audit, the two cannot be distinguished.

89 **Retrieval.** The bet is that recurrent state is unnecessary: index past sessions, retrieve at query time,
90 append to context. Examples: RAG [16], the kNN variant of Memorising Transformers [8], and
91 DOS-RAG [17]. This program is exposed to *window leakage* (FM4, §5.4) by construction, since
92 retrieved evidence enters the LM attention window deliberately, and to surface-form/literal-match
93 shortcuts that the long-context-evaluation literature (NoLiMa [1], RULER [2], Lost-in-the-Middle [3])
94 has documented for retrieval but rarely connected back to memory architectures.

95 **The long-context evaluation cluster.** The closest neighbour to this paper’s methodology is the
96 long-context-evaluation literature. NoLiMa [1] removes literal overlap between query and evidence;
97 RULER [2] adds UUIDs and hard distractors; Lost-in-the-Middle [3] documents position effects;
98 LongBench [4] introduces no-context baselines; LongMemEval [6] separates indexing, retrieval, and
99 reading; BABILong [5] embeds bAbI facts in PG19 distractors; ZeroSCROLLS [19] contributes
100 contamination-controlled zero-shot evaluation. Geirhos et al. [7] provide the broader frame: shortcut
101 learning is the default when the benchmark permits it. This cluster has imported anti-shortcut
102 techniques from the benchmark-design literature but has not yet exported them back into recurrent-
103 memory architecture evaluation; one contribution of this paper is precisely that re-export.

104 The pattern in Table 1 is the field-level finding. None of the three programs is self-validating. An
105 honest evaluation of recurrent memory has to rule out the failure modes its own ideology is blind to
106 and the failure modes the other two ideologies are sensitive to. The contribution of this paper is to
107 consolidate the controls into a single battery that survives across all three.

3 Memory Residuals: A Cross-Regime Substrate

We use Memory Residuals as a substrate, not as a proposed method. It is, to our knowledge, the only recurrent-memory architecture that admits all three ideological framings on the same parameter count and same corpus, which is what makes the cross-regime comparison in this paper possible.

State. A fixed-size memory $M_c \in \mathbb{R}^{K \times d}$ with $K = 128$ slots and d the backbone hidden size. M_c is updated once per session via a two-stage Extract+Judge slot-attention writer and read back into the LM through a depth-routed cross-attention residual.

Extract. Given a session token-context representation $C_t \in \mathbb{R}^{N_t \times d}$ from a fixed backbone layer (hidden_14 with input RMSNorm in our setup), a learned query bank M_{in} cross-attends to produce a candidate memory contribution:

$$M_{\text{new}} = \text{softmax} \left(\frac{M_{\text{in}} W_Q^{\text{ext}} (C_t W_K^{\text{ext}})^{\top}}{\sqrt{d}} \right) C_t W_V^{\text{ext}}. \quad (1)$$

Judge. A slot-attention writer with 3 iterations, orthogonal query initialisation, slot positional embeddings, and judge QK layer-norm combines the previous state M_c^{t-1} with the candidate M_{new} :

$$M_c^t = \text{Slot}_{\theta}([M_c^{t-1} \parallel M_{\text{new}}]). \quad (2)$$

Read. At inference, token representations X at each routed sublayer cross-attend to M_c to produce a memory readout m_t :

$$m_t = \text{softmax} \left(\frac{X W_Q^r (M_c W_K^r)^{\top}}{\sqrt{d}} \right) M_c W_V^r. \quad (3)$$

The readout m_t is registered as an additional source in a Block-Attention-Residual router; a learned routing mass $\alpha_{\text{mem}} \in [0, 1]$ controls the share of LM computation that uses memory at each routed sublayer. Two architectural details turn out to be load-bearing in the surviving recipe (§8): an iterative readout depth (re-applying the readout cross-attention $L_R=4$ times before residual injection) and a learned α_{mem} -floor that prevents $\alpha_{\text{mem}} \rightarrow 0$ collapse during training.

Three regimes on the same architecture. The same code path supports three ideological regimes by changing only the -freeze_backbone, -readout_probe_loss_weight, and -retrieval_index flags. All six failure modes catalogued in §5 are exhibited on this single architecture under the appropriate regime, which is what allows the cross-regime comparison without confounding architecture and ideology.

4 Experimental Setup

Architecture. All experiments use Memory Residuals (§3) attached to two frozen pretrained backbones: Qwen3-0.6B and Qwen3-1.7B. Only the slot-attention writer and the depth-routed readout are trained; the LM head and all backbone parameters are held fixed throughout. Trainable parameter count: 41.5M at 0.6B ($\sim 6\%$ overhead) and 164.8M at 1.7B ($\sim 8\%$). Full hyperparameters are reported in Appendix A.

Corpora. The campaign uses three corpora at different stages. The *synthetic persona corpus* (campaign label D4v2) consists of multi-session chats where each chain establishes a closed-set persona attribute (e.g. a favourite colour drawn from 32 categories) in an evidence session and queries it via a templated callback (“What was my favorite COLOR?”). It is the corpus most exposed to the template-prior leak (FM3) and to window leakage at small window_k (FM4). The *random-code corpus* (D5) replaces closed-set attributes with per-chain random alphanumeric codes and masks the LM loss on evidence sessions, leaving the callback span as the only supervised answer location; this corpus removes the template prior and is the diagnostic for the writer-objective failure (FM6). *LongMemEval-S* [6] is the natural-text held-out evaluation corpus; we evaluate on the validation split, 50 chains.

Table 2: The six failure modes mapped to the non-memory pathway each exploits and to the ideology most exposed by construction.

#	Failure mode	Pathway	Most exposed ideology
FM1	Token-averaged dilution	Measurement	Frozen backbone
FM2	Joint-training invisibility	Backbone template prior	Joint training
FM3	Category-cue leak	Backbone template prior	Joint training; retrieval (partial)
FM4	Window leakage	Window path	All three
FM5	Redaction uncertainty	Measurement	Frozen backbone
FM6	Writer-objective failure	Content-blind memory prior	Frozen backbone

Metrics. For a chain c scored at its callback position, let $CE_{\text{mem}}(c)$ denote the cross-entropy on callback tokens under the trained model with M_c built from the chain’s prefix sessions; $CE_{\text{nomem}}(c)$ under M_c zeroed; $CE_{\text{shuffle}}(c)$ under M_c built from a different chain (the chain-shuffle confound control); $CE_{\text{floor}}(c)$ under M_c built from a chain whose evidence sessions have been replaced by distractor sessions sampled from filler positions of unrelated chains. We define

$$\begin{aligned}\Delta_{\text{nm-m}} &= CE_{\text{nomem}} - CE_{\text{mem}}, && \text{chain-specific memory benefit;} \\ \Delta_{\text{sh-m}} &= CE_{\text{shuffle}} - CE_{\text{mem}}, && \text{history-specificity confound;} \\ \Delta_{\text{evidence}} &= \Delta_{\text{nm-m}} - (CE_{\text{nomem}} - CE_{\text{floor}}), && \text{evidence-specific lift over the no-evidence floor.}\end{aligned}$$

Standard chain-CE evaluation aggregates across all tokens of the scored window. *Callback-aware* evaluation, denoted $\Delta_{\text{nm-m}}^{\text{cb}}$, restricts the score to the annotated callback answer tokens only.

Recipes and naming convention. We refer to two recipe families that differ in a single training flag. F3-ON adds an auxiliary readout-side probe loss that asks the readout to predict a chain-identifying signal directly from M_c . F3-OFF disables this auxiliary loss; only the long-path LM-NLL gradient supervises the writer and readout. In the campaign ledger, F3-ON cells are labelled v24a (0.6B) and v25a (1.7B); F3-OFF cells are v27b (0.6B) and v28 (1.7B). All other architectural and optimisation details match across the two recipes; the only flag that varies is `-readout_probe_loss_weight`.

Compute and seeds. Each 0.6B run consumes approximately 1.5 H100-hours; each 1.7B run approximately 6 H100-hours. The headline F3-OFF flip is established at $n = 4$ seeds at 0.6B and $n = 2$ seeds at 1.7B, with their F3-ON counterparts at $n = 3$ and $n = 2$ respectively. Total campaign compute, including unreported preliminary runs from earlier ablations, is approximately 400 H100-hours on H100 80GB hardware.

Statistical testing. Headline numerical claims are accompanied by bootstrap 95% confidence intervals over the 50 LongMemEval-S validation chains. The F3-OFF flip is certified by a chain-level paired sign-flip permutation test (10,000 permutations, one-sided) and a paired sign-test, both reported in §6 and detailed in Appendix C. We apply a Bonferroni correction across the six failure-mode claims to control the family-wise error rate.

5 Six Failure Modes

An apparent recurrent-memory result has to be ruled out from five non-memory pathways: a *content-blind memory prior* (the writer emits a similar M_c on every chain and the readout shifts the LM toward a marginal callback distribution); a *backbone template prior* (an unfrozen LM head learns the closed-set callback distribution directly from training data with M_c unused); a *window path* (evidence remains in the local LM attention window at scoring time); and two *measurement pathways* (token-averaging dilutes a localised callback gain, and an unaudited redaction definition makes a zero-lift floor look leaky or vice versa). Each pathway corresponds to one or two of the failure modes catalogued below; the mapping is summarised in Table 2.

The six failures are independent: fixing one does not fix the others, which we verify in §7 via the audit-battery composition argument. Numerical anchors throughout this section come from F3-ON checkpoints unless otherwise stated.

183 5.1 FM1. Token-averaged dilution

184 Chain-level cross-entropy averages over the entire scoring window, but chain-specific information is
 185 concentrated in the *callback answer tokens* that comprise only 1–2% of a window. A frozen-backbone
 186 F3-ON checkpoint at an early-campaign snapshot improves callback-aware CE by $\Delta_{nm-m}^{cb} =$
 187 $+1.44$ nats while *worsening* chain-level CE by $\Delta_{nm-m} = -0.085$ nats: the noise on the remaining
 188 98% of tokens dominates the mean. The diagnostic is to score CE only on the annotated answer mask
 189 (*callback-aware CE*, `tools/eval_callback.py`). This failure is most acute in the frozen-backbone
 190 program, where chain-level CE is the conventional headline metric; joint-training and retrieval papers
 191 escape it by reporting task accuracy, but they pay for that escape with their own failure modes (FM2,
 192 FM3).

193 5.2 FM2. Joint-training invisibility

194 A joint-trained backbone collapses $\Delta_{evidence}$ to approximately zero even on corpora where the
 195 frozen-backbone checkpoint shows positive callback gain. Joint training opens a non-memory path
 196 through the LM head; gradient descent allocates credit to whichever path is shortest, and the LM-head
 197 path is shorter than the $M_c \rightarrow \text{readout} \rightarrow \text{LM-head}$ path. On the synthetic persona corpus, a
 198 joint-trained 0.6B run reaches callback CE within 0.06 nats of the empirical 32-way category prior
 199 (2.91 nats per callback-mask token), and $\Delta_{evidence} \approx 0$: the LM head learned the prior, M_c is unused.
 200 The diagnostic is to compare frozen-backbone and joint-training versions of the same architecture
 201 at matched data, and to report the empirical prior CE explicitly. If joint matches the prior within
 202 0.1 nats and frozen does not, the joint result is “the LM learned the prior”, not “the memory learned
 203 the binding”.

204 5.3 FM3. Category-cue leak / template prior

205 Synthetic callback corpora that repeat a closed-set category in the user question (“What was my
 206 favorite COLOR again?”) collapse the answer distribution to a sharp, easily-fit prior. The empirical
 207 $P(\text{item} \mid \text{category})$ on the synthetic persona corpus has CE 2.9087 nats per callback-mask token, so
 208 any joint-trained baseline within ~ 0.1 nats of 2.91 is consistent with “LM head fits the prior” with
 209 no memory contribution required. The diagnostic is a Laplace-smoothed empirical prior reported
 210 explicitly as a baseline (`tools/audit_a3_template_prior.py`). Joint-training papers are most
 211 exposed; retrieval is partially exposed because the LM head still fits the prior even with retrieved
 212 context; frozen-backbone is immune by construction *provided* the LM head is truly frozen — a
 213 non-trivial provision because some adapter-style fine-tuning leaks unfrozen capacity.

214 5.4 FM4. Window leakage

215 Evidence sessions can remain inside the LM attention window at callback scoring time even when a
 216 corpus declares them “out of window”. On the synthetic persona corpus at the default `window_k=4`,
 217 58.5% of training chains and 59.6% of validation chains have at least one evidence session inside
 218 the callback scoring window: memory may appear to help only because evidence is in-window. The
 219 diagnostic is to count the per-chain leak fraction (`tools/audit_a1_window_leakage.py`) and
 220 report it for the chosen `window_k`. Retrieval relies on this path deliberately, so retrieval papers should
 221 disclose the leak count rather than wishing it away; frozen and joint backbones are exposed unless
 222 the corpus enforces a strict “evidence outside window” gap.

223 5.5 FM5. Redaction uncertainty

224 The frozen-backbone evidence-redacted floor underpins the $\Delta_{evidence} = \Delta_{nm-m} - (\text{CE}_{\text{nomem}} -$
 225 $\text{CE}_{\text{floor}})$ measurement. But $\Delta_{evidence} \approx 0$ admits two interpretations: (a) the writer is not encoding
 226 chain-specific information (real null), or (b) the redaction is leaky and M_c built from “redacted” data
 227 still contains the answer (false null). The diagnostic is to compare four redaction variants (default,
 228 cross-chain, skip-evidence, zeroed-token) and require an inter-variant spread ≤ 0.05 nats before
 229 trusting an $\Delta_{evidence} \approx 0$ reading (`tools/audit_a2_redaction.py`). On a representative F3-ON
 230 0.6B checkpoint the spread is 0.038 nats and the default floor is slightly worse than the cross/zero
 231 variants, so the floor is not leaking in the dangerous direction. The diagnostic costs $4\times$ a single
 232 Δ_{nm-m} evaluation.

Table 3: F3-OFF vs F3-ON evaluated on the same 50 LongMemEval-S validation chains. The only flag that varies between F3-OFF and F3-ON is `-readout_probe_loss_weight`. $\Delta_{\text{sh-m}}$ remains within ± 0.02 nats throughout (chain-specificity preserved); Δ_{evidence} is statistically zero, supporting the chain-conditional-context interpretation (§8). Campaign labels: F3-ON cells are v24a (0.6B) and v25a (1.7B); F3-OFF cells are v27b (0.6B) and v28 (1.7B).

Recipe	Scale (n seeds)	$\Delta_{\text{nm-m}}$ (nats)	$\Delta_{\text{sh-m}}$ (nats)	Δ_{evidence}
F3-ON	0.6B ($n = 3$)	$+0.162 \pm 0.083$	$+0.007 \pm 0.008$	$+0.005 \pm 0.003$
F3-OFF	0.6B ($n = 4$)	$+1.323 \pm 0.530$	$+0.000 \pm 0.010$	$+0.001 \pm 0.006$
F3-ON	1.7B ($n = 2$)	$+0.118$	$+0.003$	-0.009
F3-OFF	1.7B ($n = 2$)	$+0.926$	-0.005	-0.001

5.6 FM6. Writer-objective failure

A random-code corpus (per-chain alphanumeric ID, evidence-session LM loss masked, no category cue) removes the template prior of FM3 and the partial leak of FM4 — and yet Δ_{evidence} remains near zero. An F3-ON 0.6B run on the random-code corpus records $\Delta_{\text{evidence}} = -0.0024$ nats early in training and at most $+0.011$ at later checkpoints, with $\Delta_{\text{nm-m}}^{\text{cb}} = -0.076$ and $\alpha_{\text{mem}} \approx 0.01$. The cause is not data; it is a writer-side optimisation problem. The writer receives gradient only through the long path callback $\text{NLL} \rightarrow \text{LM head} \rightarrow \text{depth router} \rightarrow \text{readout} \rightarrow M_c \rightarrow \text{writer}$; without a category prior, “wrong code” and “uninformative memory” both look bad to the LM loss, so there is no direct extractive signal. The diagnostic is a *test-time readout audit*: freeze all parameters except the readout’s W_Q, W_K, W_V and TTT-optimize on callback tokens of held-out chains. If the resulting CE drops below the floor by more than 0.30 nats, the writer encoded the answer and the readout was the bottleneck; if not, the writer never encoded the literal answer (`tools/eval_ttt_mc_readout.py`). Frozen-backbone is most exposed; any architecture whose memory module is trained only by long-path gradient through the LM head is at risk. Retrieval is exempt because it has a separate index-time objective.

6 Headline Result: The Auxiliary Readout Probe Backfires

The natural fix for the writer-objective failure (FM6) is the auxiliary readout-side probe loss — the F3 probe of §4 — that asks the readout to predict a chain-identifying signal directly from M_c . F3-ON yields modest gains: $\Delta_{\text{nm-m}} = +0.16 \pm 0.08$ nats at 0.6B ($n = 3$ seeds) and $+0.12$ nats at 1.7B ($n = 2$ seeds). Switching the F3 probe off — the only flag change being the auxiliary-loss weight — multiplies $\Delta_{\text{nm-m}}$ by $\sim 8\times$ at both scales (Table 3): $+1.32 \pm 0.53$ nats at 0.6B ($n = 4$ seeds) and $+0.93$ nats at 1.7B ($n = 2$ seeds). Two single-variable ablations (Table 4) confirm load-bearing components of the surviving recipe: removing the iterative readout depth collapses $\Delta_{\text{nm-m}}$ to $+0.025$, and removing the α_{mem} -floor drives it to -0.04 . Removing the F3 probe alone, by contrast, *strengthens* the result.

Chain-level paired permutation test. The seed-level standard deviation at $n = 4$ (0.530 nats) is large relative to the mean. We therefore certify the effect against a chain-level paired sign-flip permutation test that exploits the 50 paired chains rather than the 7 paired seeds. For each chain c define

$$\delta_c = \frac{1}{4} \sum_{s \in \text{F3-OFF}} [\text{CE}_{\text{nomem}}^{(s)}(c) - \text{CE}_{\text{mem}}^{(s)}(c)] - \frac{1}{3} \sum_{s \in \text{F3-ON}} [\text{CE}_{\text{nomem}}^{(s)}(c) - \text{CE}_{\text{mem}}^{(s)}(c)]. \quad (4)$$

Under H_0 (F3 has no causal effect on chain-level memory benefit), F3-ON/F3-OFF labels are exchangeable per chain, so $\text{sign}(\delta_c)$ is i.i.d. ± 1 . The observed mean is $+1.16$ nats per chain (95% bootstrap CI $[+0.84, +1.52]$); zero of 10,000 sign-flip permutations matched its magnitude (one-sided $p < 10^{-4}$, Figure 1 left). The effect is not driven by a few chains: 47 of 50 chains favour F3-OFF (one-sided sign-test $p = 1.9 \times 10^{-11}$), the per-chain median is $+0.60$ nats, and the worst chain is within 0.07 nats of zero (Figure 1 right). Each of the twelve cross-seed pairwise comparisons has F3-OFF winning on at least 42 of 50 chains (range 42–49); the 1.7B replication records 44/50 chains positive at each individual seed.

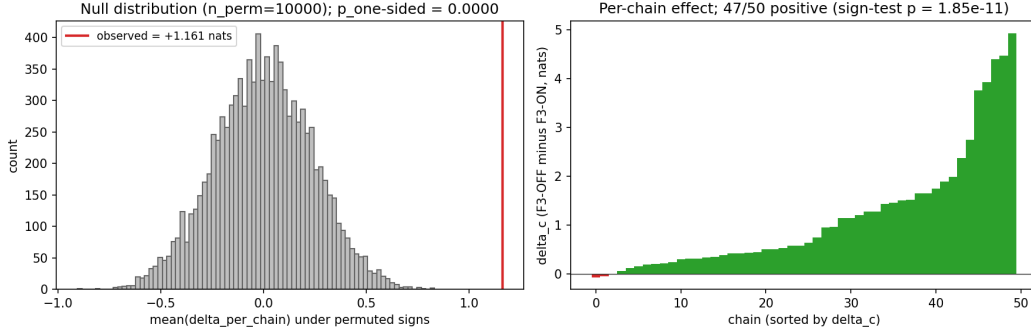


Figure 1: Chain-level paired permutation test for the F3-OFF flip, across all four F3-OFF 0.6B seeds and three F3-ON 0.6B seeds, evaluated on the same 50 LongMemEval-S validation chains under the canonical `eval_callback.py` pipeline. Left: null distribution of mean δ_c over 10,000 paired sign-flip permutations; observed value +1.16 nats marked in red. Right: per-chain δ_c sorted; 47 of 50 chains favour F3-OFF (green), 3 favour F3-ON (red), median +0.60 nats.

Why the F3 probe hurts. The F3 probe asks the readout to recover a chain-identifying signal directly from M_c . Joint LM-NLL training alone produces a richer chain-conditional context than supervising the readout for chain identification: the probe’s gradient pulls the readout toward a chain-ID decoder, while the LM-NLL gradient asks the readout to compress the chain into a context vector that lowers next-token loss on *any* future callback in the chain. Removing the probe lets the latter dominate. $\Delta_{sh-m} \approx 0$ across all six F3-OFF seeds confirms the resulting M_c is chain-specific; the small $\Delta_{evidence}$ confirms that the writer is compressing chain-conditional context (style, topic, vocabulary, prior turns) rather than memorising the literal answer-bearing session. This makes the F3 prescription a falsified ideology-level intervention: auxiliary readout supervision is the standard fix proposed in the joint-training literature for FM6 and is empirically shown here to backfire.

7 Six-Item Audit Methodology

The audit battery is the constructive output of the campaign: a single set of controls that any recurrent-memory paper can run in under one H100-hour per checkpoint, regardless of ideology.

- Callback-aware CE (FM1).** Restrict scoring to annotated callback answer tokens. Released as `tools/eval_callback.py`.
- Evidence-redacted memory floor (FM5, FM6).** Build M_c from a chain whose evidence sessions are replaced by distractor sessions sampled from filler positions of other chains; the resulting Δ_{nm-m} is the no-evidence floor that $\Delta_{evidence} = \Delta_{nm-m} - (CE_{nomem} - CE_{floor})$ measures against. The floor itself is validated by comparing default, cross-chain, skip-evidence, and zeroed-token redaction variants; require ≤ 0.05 nat spread between variants before trusting an $\Delta_{evidence} \approx 0$ reading (`tools/audit_a2_redaction.py`).
- Empirical and base-model prior audits (FM2, FM3).** Build a Laplace-smoothed $P(\text{item} | \text{category})$ prior over the corpus and a base-model callback CE without memory or fine-tuning. Memory results that do not beat both priors are not memory results. Released as `tools/audit_a3_template_prior.py` and `tools/audit_base_prior.py`.
- Window-leakage count (FM4).** Count the fraction of chains where the gold evidence session falls inside the callback scoring window at the chosen `window_k`. Released as `tools/audit_a1_window_leakage.py`.
- Test-time readout audit (FM6).** TTT only the readout W_Q, W_K, W_V on callback tokens of held-out chains; if CE drops below the floor by more than 0.30 nats, the writer encoded the answer and the readout was the bottleneck; if not, the writer is the bottleneck. Released as `tools/eval_ttt_mc_readout.py`. Numerical results on the surviving recipe are reported in Appendix D; all 7 checkpoints under all 3 TTT modes return “writer is the bottleneck”, supporting the chain-conditional-context interpretation.

Table 4: Single-variable ablations on the F3-ON 0.6B recipe at seed 1. “final” is end-of-training; “best” is best by evidence_lift. All cells differ from F3-ON-seed-1 by exactly one CLI flag. Campaign labels in parentheses.

Ablation	Run	Δ_{nm-m} (final)	Δ_{nm-m} (best)	Verdict
F3-ON baseline	v24a-seed1	+0.227	+0.229	reference
– readout depth	v27a	+0.025	+0.029	load-bearing — keep
– α_{mem} -floor	v27c	−0.038	−0.289	load-bearing — keep
– F3 readout probe	v27b	+0.797	—	harmful — drop

6. **Held-out-corpus pivot.** Report results on a corpus that the architecture has not been corpus-tuned against and that differs qualitatively from the training corpus (e.g. a synthetic closed-set persona corpus $\rightarrow$ a natural multi-session dialogue corpus). In our campaign, results that looked positive on the synthetic persona corpus for the wrong reasons (FM3 + FM4) collapsed to honest numbers on LongMemEval-S without recipe changes. This is the cross-corpus analogue of the per-checkpoint audits and is necessary for ruling out template-corpus confounds that no single per-checkpoint diagnostic catches.

Why the six are independent. Each audit closes a distinct non-memory pathway (Table 2); fixing one pathway does not block the others. Token-averaging (FM1) lives in the metric, the joint-training prior (FM2/FM3) in the architecture choice, the window leak (FM4) in the corpus, the redaction ambiguity (FM5) in the floor definition, the writer collapse (FM6) in the optimisation regime, and the held-out-corpus pivot in the cross-corpus experiment design. Six different surfaces; six different diagnostics.

8 The Recipe That Survived All Six Audits

Applying the audit battery prospectively across the campaign, exactly one configuration survives all six audits with positive chain-specific evidence: a frozen Qwen3 backbone, a fixed-size slot-attention M_c with $K = 128$, depth-routed readout with iterative readout depth $L_R=4$, an α_{mem} -floor on the depth router, and *no* auxiliary writer or readout probe. In campaign labels this is v27b at 0.6B and v28 at 1.7B; the ablation table (Table 4) shows the load-bearing components are the readout depth and the α_{mem} -floor; the canonically prescribed F3 readout probe is, as §6 reports, harmful.

Headline survival numbers. Memory-on vs. memory-off callback CE on 50 LongMemEval-S validation chains under the surviving recipe: $\Delta_{nm-m} = +1.32 \pm 0.53$ nats at 0.6B ($n = 4$ seeds), $\Delta_{nm-m} = +0.93$ nats at 1.7B ($n = 2$ seeds, both at $+0.91/ +0.94$). Chain-shuffle confound $\Delta_{sh-m} = 0.000 \pm 0.010$ across all four 0.6B seeds and both 1.7B seeds; evidence-lift $\Delta_{evidence} = +0.001 \pm 0.006$, statistically zero.

Interpretation. $\Delta_{sh-m} \approx 0$ rules out a non-chain-specific cause; the LM-on-LM-off gain is genuinely conditioned on the chain. $\Delta_{evidence} \approx 0$ on a frozen-backbone recipe is unusual: it says the writer is not encoding the literal evidence session into M_c . The held-out TTT-readout audit (§7 item 5, Appendix D) confirms this directly: across all four 0.6B and three 1.7B F3-OFF checkpoints, in all three TTT modes, the test-time-trained readout *cannot* recover the answer from M_c — mean lift over the floor is between -0.66 and -1.07 nats depending on mode and scale, where the verdict threshold is $+0.30$. The writer never encoded the literal answer. M_c is therefore best understood as encoding chain-conditional context — style, topic, vocabulary, prior turns — that shifts the LM toward better priors on continuations of the chain, rather than as encoding literal evidence for retrieval. This is a different mechanism from the “memory recall” the joint-training and retrieval programs target, and it is consistent with the modest Δ_{nm-m} we observe relative to oracle-RAG: the surviving recipe is a chain-context conditioner, not a fact memorizer.

9 Limitations

We separate the limitations of this paper into three axes: scope, architectural class, and methodological coverage.

Scope. Our results are reported on Qwen3-0.6B and Qwen3-1.7B and on a single architectural family (slot-attention writer, depth-routed cross-attention readout). The F3-OFF flip is established at $n = 4$ seeds at 0.6B and $n = 2$ seeds at 1.7B; behaviour at scale ($\geq 7B$) and on non-slot writer architectures (NTM-style external memory, Perceiver-IO-style fixed-bandwidth latents) is open. We test one corpus family for the writer-objective failure (the random-code corpus) and report the held-out M_c -probe behaviour on the LongMemEval-S validation split, 50 chains; LoCoMo [20] and natural multi-week dialogue corpora remain future work.

Architectural-level versus measurement-level claims. Four of the six failure modes (FM1, FM3, FM4, FM5) are evaluation-level: they live in the metric, the corpus, or the floor definition, and generalise to any recurrent-memory paper using the affected metric or corpus regardless of architecture. Two (FM2 and FM6) are architecture-level. We state the architectural class in each case and exhibit the failure on Memory Residuals at $\leq 1.7B$; we do not claim, e.g., that all joint-trained architectures collapse Δ_{evidence} to zero in all training regimes, only that this is the failure mode the joint-training program must rule out and that a published-style architecture in the program has been observed to exhibit it (FM2 example).

Sufficiency of the audit battery. We do not claim the six-item audit battery is *sufficient* to certify a recurrent-memory result. We claim it is *necessary*: each of the six controls catches a failure that is independently load-bearing on at least one ideology, and the union catches all failures we have observed in the campaign. New corpora and new architectures may surface new failure modes the battery does not capture.

The F3-OFF surprise. We do not claim auxiliary readout-side probe losses are universally harmful. We tested one writer family (slot-attention) at $\leq 1.7B$ with a chain-identification probe target. The effect at scale, with a different probe target (e.g. callback-content prediction rather than chain ID), or in non-slot writer architectures is open. The chain-conditional-context interpretation of $\Delta_{\text{evidence}} \approx 0$ on the surviving recipe is supported by the held-out TTT-readout audit (Appendix D); a positive result on a stronger writer-side extractive objective would re-open the question.

Cross-architectural validation. The claim that the field-level reporting matrix in Table 1 systematically under-reports controls is supported by a literature audit of 18 representative papers (Appendix E); we do not claim it is exhaustive, and individual papers may have addressed a control in a way the audit misses. The matrix is intended as a structured argument, not as a final scoring of the field.

10 Conclusion

The conservative answer to “when does recurrent memory help?” is “when no other path can solve the proxy”. Across the campaign, that condition factored into six independent failure modes that map onto three ideological research programs. The six-item audit methodology (§7) certifies a single recipe — frozen backbone, slot-attention M_c , depth-routed readout, α_{mem} -floor, and *no* auxiliary readout probe — that survives all six. The F3-OFF empirical surprise refutes the joint-training prescription that auxiliary writer/readout supervision is needed for FM6: the LM-NLL signal is sufficient when the architectural constraints (frozen backbone, α_{mem} -floor, readout depth) are in place. The TTT-readout audit on the surviving recipe (Appendix D) further refines the interpretation: M_c encodes chain-conditional context, not literal evidence. Future work the methodology points to includes scaling the surviving recipe past 1.7B, testing the F3-OFF effect under non-slot writer families, and extending the audit battery to natural long-term memory corpora such as LoCoMo [20].

Acknowledgments

Acknowledgments are deferred for double-blind review.

Reproducibility Statement

All numbers in the main text are produced by a single locked eval script (tools/eval_callback.py) on the patched LongMemEval-S validation corpus (paper_artifacts/chains/lme_val_s512_evpos.pt, 50 chains). The seven seeds (3 v24a + 4 v27b at 0.6B) and the 1.7B v25a/v28 cells were launched from Scripts/train_v{24a,25a,27b,28}*.sh; only the -readout_probe_loss_weight flag varies between F3-ON and F3-OFF. The chain-level paired permutation test is implemented at tools/permutation_test_f3_canonical.py and run against the canonical eval JSONs in results/eval_v25_seed_pack_evpos/. The TTT-readout audit is launched from Scripts/run_ttt_mc_readout_v27b_v28.sh; the per-cell JSON outputs and aggregate table are in results/ttt_mc_v27b_v28/. The audit-battery tools are released in the supplementary zip and listed in §7. Hyperparameter table: Appendix A.

References

- [1] Modarressi et al. NoLiMa: Long-Context Evaluation Beyond Literal Matching. arXiv:2502.05167.
- [2] Hsieh et al. RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.
- [3] Liu et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172.
- [4] Bai et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508.
- [5] Kuratov et al. BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack. arXiv:2406.10149.
- [6] Yu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813.
- [7] Geirhos et al. Shortcut Learning in Deep Neural Networks. arXiv:2004.07780.
- [8] Wu et al. Memorizing Transformers. arXiv:2203.08913.
- [9] Bulatov, Kuratov, and Burtsev. Recurrent Memory Transformer. arXiv:2207.06881.
- [10] Rodkin et al. Associative Recurrent Memory Transformer. ICML 2024 Next Generation of Sequence Modeling Architectures workshop.
- [11] Chevalier et al. Adapting Language Models to Compress Contexts. arXiv:2305.14788.
- [12] Zhong, Lei, and Chen. Training Language Models with Memory Augmentation. arXiv:2205.12674.
- [13] Hutchins et al. Block-Recurrent Transformers. NeurIPS 2022.
- [14] Rae et al. Compressive Transformers for Long-Range Sequence Modelling. arXiv:1911.05507.
- [15] Graves, Wayne, and Danihelka. Neural Turing Machines. arXiv:1410.5401.
- [16] Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- [17] Laitenberger et al. Stronger Baselines for Retrieval-Augmented Generation with Long-Context Language Models (DOS-RAG). arXiv:2506.03989.
- [18] Goyal et al. Recurrent Independent Mechanisms. ICLR 2021. arXiv:1909.10893.
- [19] Shaham et al. ZeroSCROLLS: A Zero-Shot Benchmark for Long Text Understanding. EMNLP Findings 2023. arXiv:2305.14196.
- [20] Maharana et al. Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). arXiv:2402.17753.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 637 • The paper should disclose whether the full research project required more compute
638 than the experiments reported in the paper (e.g., preliminary or failed experiments that
639 didn't make it into the paper).

640 9. Code of ethics

641 Question: Does the research conducted in the paper conform, in every respect, with the
642 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

643 Answer: **[TODO]**

644 Justification: **[TODO]**

645 Guidelines:

- 646 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
647 Ethics.
- 648 • If the authors answer [No], they should explain the special circumstances that require a
649 deviation from the Code of Ethics.
- 650 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
651 eration due to laws or regulations in their jurisdiction).

652 10. Broader impacts

653 Question: Does the paper discuss both potential positive societal impacts and negative
654 societal impacts of the work performed?

655 Answer: **[TODO]**

656 Justification: **[TODO]**

657 Guidelines:

- 658 • The answer [N/A] means that there is no societal impact of the work performed.
- 659 • If the authors answer [N/A] or [No], they should explain why their work has no societal
660 impact or why the paper does not address societal impact.
- 661 • Examples of negative societal impacts include potential malicious or unintended uses
662 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
663 (e.g., deployment of technologies that could make decisions that unfairly impact specific
664 groups), privacy considerations, and security considerations.
- 665 • The conference expects that many papers will be foundational research and not tied
666 to particular applications, let alone deployments. However, if there is a direct path to
667 any negative applications, the authors should point it out. For example, it is legitimate
668 to point out that an improvement in the quality of generative models could be used to
669 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
670 that a generic algorithm for optimizing neural networks could enable people to train
671 models that generate Deepfakes faster.
- 672 • The authors should consider possible harms that could arise when the technology is
673 being used as intended and functioning correctly, harms that could arise when the
674 technology is being used as intended but gives incorrect results, and harms following
675 from (intentional or unintentional) misuse of the technology.
- 676 • If there are negative societal impacts, the authors could also discuss possible mitigation
677 strategies (e.g., gated release of models, providing defenses in addition to attacks,
678 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
679 feedback over time, improving the efficiency and accessibility of ML).

680 11. Safeguards

681 Question: Does the paper describe safeguards that have been put in place for responsible
682 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
683 image generators, or scraped datasets)?

684 Answer: **[TODO]**

685 Justification: **[TODO]**

686 Guidelines:

- 687 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Architecture and Training Details

Architectural hyperparameters. Memory size $K = 128$ slots at backbone hidden width d ; extract source layer hidden₁₄ with input RMSNorm; slot-attention judge with 3 iterations, orthogonal query initialisation, slot positional embeddings, and judge QK layer-norm; iterative readout depth $L_R = 4$; depth-routed Block-Attention-Residual readout with learned α_{mem} -floor. Trainable parameter count 41.5M at Qwen3-0.6B ($\sim 6\%$ overhead) and 164.8M at Qwen3-1.7B ($\sim 8\%$).

Training hyperparameters. Optimiser AdamW with $\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1; cosine LR schedule with 200 warmup steps and 5% minimum ratio; learning rates $\eta_{M_c} = 3 \times 10^{-4}$ for memory-side parameters and $\eta_{LM} = 0$ (frozen). Batch size 4 chains with gradient accumulation 4 (effective 16); 1000 optimiser steps; bf16 precision with gradient checkpointing.

789 **Compute.** Each 0.6B run uses approximately 1.5 H100-hours on a single H100 80GB; each 1.7B
790 run approximately 6 H100-hours. Total campaign compute, including unreported preliminary runs
791 from earlier ablations, is approximately 400 H100-hours.

792 B Audit Numbers

793 **A1 window leakage.** At `window_k = 4`, 2926/5000 training chains and 298/500 validation chains
794 leak; at $k = 3$, 2068/5000 and 217/500; at $k = 2$, 1088/5000 and 114/500; at $k = 1$, zero.

795 **A2 redaction validity.** Representative F3-ON 0.6B checkpoint (best-by-elift): default 5.1235, cross-
796 chain 5.0911, skip-evidence 5.0857, zeroed-token 5.0896 (spread 0.038 nats, within the pre-declared
797 0.05-nat band). An F3-ON 0.6B joint-training counterpart shows spread 0.0066 nats.

798 **A3 template prior.** Empirical $P(\text{item} \mid \text{category})$ CE on 128 validation chains = 2.9087 nats;
799 F3-ON joint-training 0.6B run reaches best 2.9655, final 2.9394 (within 0.06 and 0.03 nats of the
800 prior).

801 **Base callback CE without memory or fine-tuning.** Qwen3-0.6B base 7.686, Qwen3-1.7B base
802 8.764; uniform over 256-item closed set 5.545 nats; uniform over the cued 32-item category
803 3.466 nats; empirical per-token template prior 2.909 nats.

804 C Permutation Test Details

805 The chain-level paired permutation test runs over the 50 LongMemEval-S vali-
806 dation chains shared across all 7 relevant checkpoints (3 F3-ON v24a + 4 F3-
807 OFF v27b at 0.6B). 10,000 sign-flip permutations; rng seed 2026. Result file:
808 `results/exp2_chain_recipe/permutation_test_canonical/permutation_test_results.json`.
809 Aggregate result: observed $T = +1.16$ nats per chain, 95% bootstrap CI $[+0.84, +1.52]$, one-sided
810 permutation $p < 10^{-4}$, sign-test $p = 1.9 \times 10^{-11}$. Per-chain bar plot in `permutation_test.png`.

811 D TTT-Readout Audit on the Surviving Recipe

812 The TTT-readout audit (item 5 of the methodology, §7) is run on the four F3-
813 OFF v27b 0.6B checkpoints and the three F3-OFF v28 1.7B checkpoints via
814 `Scripts/run_ttt_mc_readout_v27b_v28.sh`. Three TTT modes are tested per check-
815 point: `v_only` (TTT on M_c and the readout’s W_V only), `qkv` (TTT on M_c and all three readout
816 projections W_Q, W_K, W_V), and `qkv_reset` (as `qkv` but the readout W_Q, W_K, W_V are re-initialised
817 from $\mathcal{N}(0, d^{-1/2})$ before TTT, testing intrinsic readout capacity from a clean init). Each cell uses
818 $K = 80$ Adam steps with $\eta_{M_c} = 10^{-2}$, $\eta_{\text{readout}} = 10^{-3}$, $n = 16$ held-out chains. The per-chain
819 decision metric is $\text{lift}_{\text{floor}} = \text{CE}_{\text{floor}} - \text{CE}_{\text{ttt}}$; $> +0.30$ nats indicates the writer encoded the answer
820 and the readout was the bottleneck; ≤ 0 indicates the writer did not encode the literal answer.

821 **Result.** Across all 7 checkpoints and all 3 modes (21 cells, 336 chain-level evaluations) the audit
822 returns the **negative** verdict (Table 5): mean lift-vs-floor between -0.66 and -1.07 nats depending
823 on mode and scale, well below the $+0.30$ nat threshold for “readout bottleneck”. Even the most
824 aggressive setting, `qkv_reset`, which re-initialises the readout from scratch and TTT-trains all three
825 projection weights jointly with M_c on the evidence-session loss, cannot recover the answer from
826 M_c . The writer never encoded the literal answer; M_c encodes chain-conditional context. The 1.7B
827 checkpoints show *larger* negative lifts than 0.6B, consistent with the surviving recipe encoding a
828 more diffuse chain-context representation at scale.

829 E Literature Audit Notes

830 This appendix supports Table 1’s cell-level judgments. We summarise, per paper, what controls are
831 reported and which of our six diagnostics are addressed; full derivations are in the campaign’s audit
832 memo `audit_b_literature.md`.

Table 5: TTT-readout audit on the F3-OFF surviving recipe. `lift` is the mean per-chain $CE_{\text{floor}} - CE_{\text{ttt}}$ across $n = 16$ held-out LongMemEval-S chains; verdict threshold is $+0.30$ for “writer-encoded” and ≤ 0 for “writer-bottleneck”. All 21 cells return “writer-bottleneck”. $\Delta_{\text{nm-m}}$ values are reported per checkpoint to confirm each cell is otherwise a positive memory result (the writer-encoded interpretation is what is being audited). Mean $\pm$ stdev are over the seeds in each scale group.

Mode	0.6B mean lift	1.7B mean lift	0.6B verdict	1.7B verdict
<code>v_only</code>	-0.66 ± 0.15 ($n = 4$)	-0.87 ± 0.26 ($n = 3$)	writer-bottleneck	writer-bottleneck
<code>qkv</code>	-0.77 ± 0.38 ($n = 4$)	-1.06 ± 0.25 ($n = 3$)	writer-bottleneck	writer-bottleneck
<code>qkv_reset</code>	-0.81 ± 0.33 ($n = 4$)	-0.97 ± 0.16 ($n = 3$)	writer-bottleneck	writer-bottleneck

Joint-training cluster. *Memorising Transformers* [8] reports natural-text LM perplexity with a memory-size ablation; no callback-aware CE, no evidence-redacted floor, no closed-book or template-prior baseline. *AutoCompressors* [11] reports memory-on/off ablations (a partial floor) but no template-prior audit and no callback-token-only CE. *TRIME* [12] uses contextualised in-batch positives/negatives and matched training/testing memory conditions; the matched conditions are a weak prior control. *RIMs* [18] and the NTM/DNC family [15] use copy/reverse/associative-recall algorithmic tasks; the algorithmic synthesis controls task identity but not category-cue priors.

Frozen-backbone cluster. *RMT* [9] reports CE on copy/reverse/associative-retrieval and on quadratic equations; no callback-aware CE or template-prior baseline. *ARMT* [10] extends RMT to associative recall with very long contexts; controls are similar. *Block-Recurrent Transformer* [13] reports PG19/arXiv/GitHub perplexity against Transformer-XL baselines; no callback-style controls. *Compressive Transformer* [14] reports natural-document perplexity; no callback-style controls.

Retrieval cluster. *RAG* [16] reports task accuracy (ODQA, fact verification); window leakage is implicit in the design but not measured as a leak. *DOS-RAG* [17] is explicitly a stronger-baselines paper; reports a closed-book baseline and discusses retrieval position effects, partial coverage of FM3 and FM4.

Long-context-evaluation cluster. *NoLiMa* [1] explicitly removes literal overlap between query and evidence (the closest analogue to our FM3) and demonstrates that scores drop sharply when the overlap is removed; counts as $\checkmark$ on Prior and on the FM3 sense of WLk. *RULER* [2] adds UUIDs and hard distractor needles, addressing FM3 (closed-set leak) and FM4 (window leakage as a function of haystack design); also $\checkmark$ on Prior and WLk. *Lost-in-the-Middle* [3] documents position effects with random-UUID K/V retrieval as a control; partial credit on Prior. *LongBench* [4] reports a closed-book/no-context baseline as a primary control; $\checkmark$ on Prior; partial credit on CB-CE because multi-task accuracy can be sensitive to per-task answer-token distribution. *BABILong* [5] embeds bAbI facts in PG19 distractors; partial control on Prior and WLk. *ZeroSCROLLS* [19] contributes contamination-controlled zero-shot evaluation; partial Prior. *LongMemEval* [6] explicitly separates indexing, retrieval, and reading; the closest existing analogue to our TTT-readout audit (item 5), counted as $\sim$ rather than $\checkmark$ because the separation is structural (different stages) rather than an interventional readout-only fine-tune.

Aggregate. Of 18 papers audited, 0 report all six diagnostics; the maximum is two diagnostics with quantitative evidence (NoLiMa, RULER), and the modal count is zero. The cluster that reports the most controls is the long-context-evaluation cluster, which has imported anti-shortcut techniques but not exported them back into recurrent-memory architecture evaluation.